> 🛠️ **2026.10 수정본 (v11)** — 원본: `ch02_03_09_NeuralNet(3).ipynb`
>
> - 윈도우 전용 폰트 경로(`C:/Windows/Fonts/...`) 대신 `koreanize-matplotlib`로 한글 폰트를 잡도록 바꿨어요. 이제 코랩·윈도우·맥 어디서나 같은 셀로 한글이 보여요. (Python 3.12부터 빠진 `distutils` 때문에 생기는 오류를 막으려고 `setuptools`도 함께 설치해요.)

In [1]:
```
### 한글
import matplotlib
# 한글 폰트 설정 (코랩·윈도우·맥 공통)  # 2026.10 수정: 윈도우 폰트 경로(C:/Windows/Fonts
%pip install -q koreanize-matplotlib setuptools
import setuptools  # 2026.10 수정: Python 3.12+엔 distutils가 없어 koreanize-ma
import koreanize_matplotlib  # 나눔고딕을 matplotlib 기본 폰트로 등록
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
import mglearn
```

L2 패널티를 사용한 모델의 복잡도 제어

- alpha를 활용한 모델의 복잡도 제어

In [3]:
```
X, y = make_moons(n_samples=100, noise=0.25, random_state=3)
print(X.shape, y.shape)
X[:5], y[:5]
```

```
(100, 2) (100,)
```

Out[3]:
```
(array([[ 1.87756309,  0.56839425],
        [ 0.36877983, -0.34894509],
        [ 0.96515318,  0.10921819],
        [ 0.48599685,  0.20291313],
        [ 1.72532644,  0.53367598]]),
 array([1, 1, 0, 1, 1]))
```

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    stratify=y, random_state=42)
```

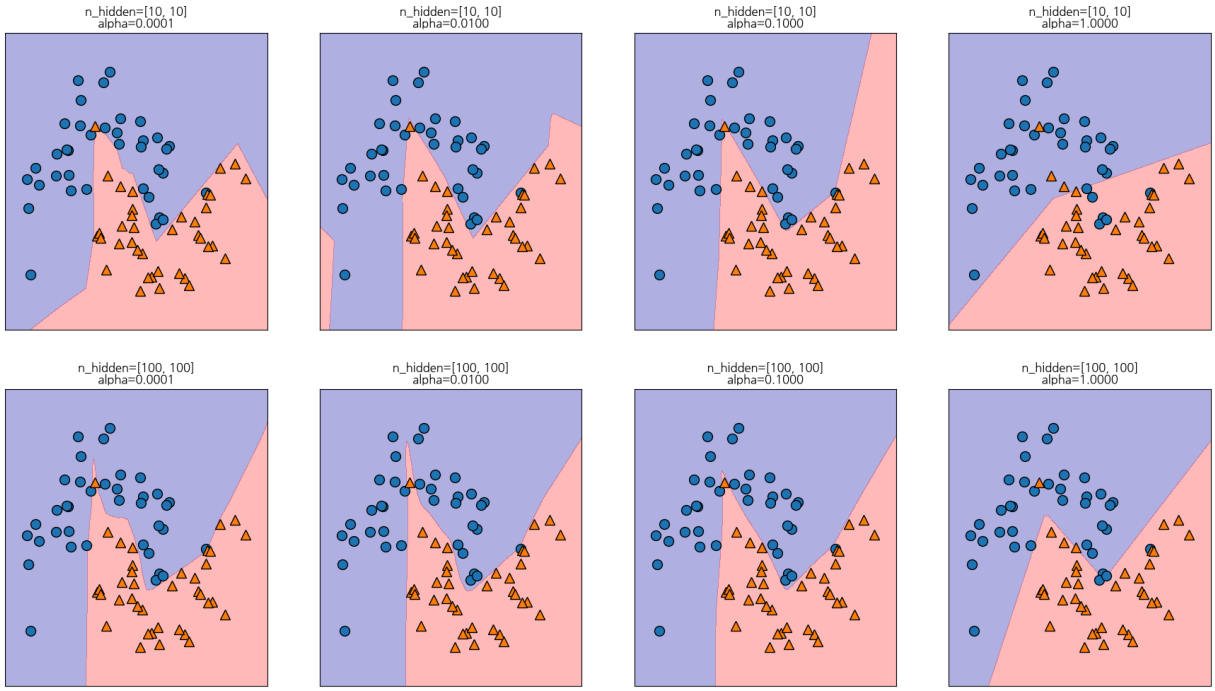
In [5]:
```
fig, axes = plt.subplots(2, 4, figsize=(22, 12))
for axx, n_hidden_nodes in zip(axes, [10, 100]):
    for ax, alpha in zip(axx, [0.0001, 0.01, 0.1, 1]):
        mlp = MLPClassifier(solver='lbfgs', random_state=0,
                            hidden_layer_sizes=[n_hidden_nodes, n_hidden_noc
                            alpha=alpha)
        mlp.fit(X_train, y_train)
        mglearn.plots.plot_2d_separator(mlp, X_train, fill=True, alpha=.3, a
        mglearn.discrete_scatter(X_train[:, 0], X_train[:, 1], y_train, ax=a
```

```
        ax.set_title("n_hidden=[{}, {}]\nalpha={:.4f}".format(
                     n_hidden_nodes, n_hidden_nodes, alpha))
```

```
.../site-packages/sklearn/neural_network/_multilayer_perceptron.py:606: Conv
ergenceWarning: lbfgs failed to converge after 200 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

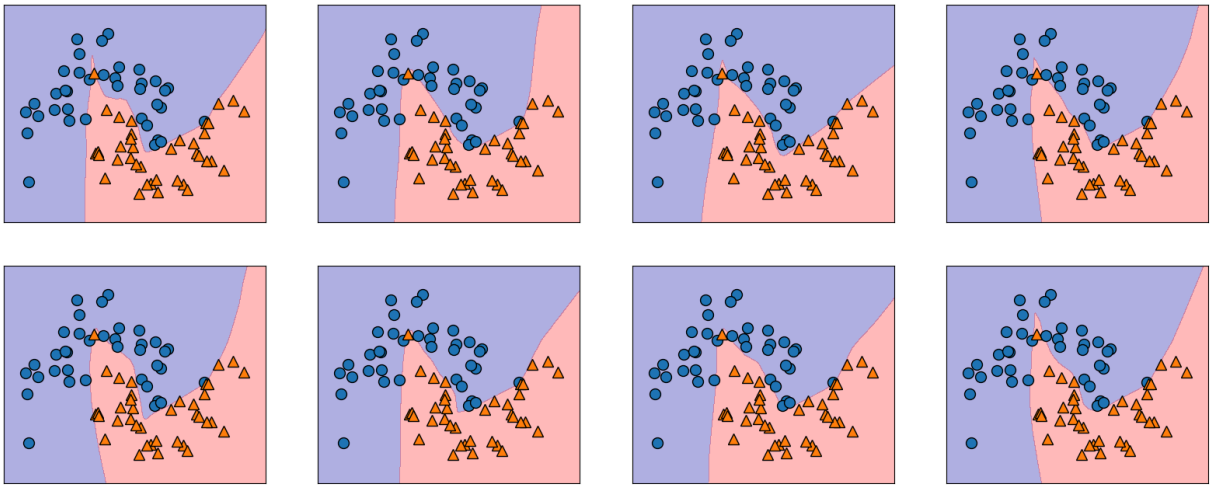
Increase the number of iterations to improve the convergence (max_iter=200).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
  self.n_iter_ = _check_optimize_result("lbfgs", opt_res, self.max_iter)
.../site-packages/sklearn/neural_network/_multilayer_perceptron.py:606: Conv
ergenceWarning: lbfgs failed to converge after 200 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=200).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
  self.n_iter_ = _check_optimize_result("lbfgs", opt_res, self.max_iter)
```



같은 매개변수이지만, 초기값이 다르면 모델이 달라진다.

In [6]:
```
fig, axes = plt.subplots(2, 4, figsize=(20, 8))
for i, ax in enumerate(axes.ravel()):
    mlp = MLPClassifier(solver='lbfgs', random_state=i,
                        hidden_layer_sizes=[100, 100])
    mlp.fit(X_train, y_train)
    mglearn.plots.plot_2d_separator(mlp, X_train, fill=True, alpha=.3, ax=ax
    mglearn.discrete_scatter(X_train[:, 0], X_train[:, 1], y_train, ax=ax)
```



유방암 데이터 셋을 활용 신경망 구현

In [7]:
```
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
```

In [8]:
```
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=0)

mlp = MLPClassifier(random_state=42)
mlp.fit(X_train, y_train)

print("훈련 세트 정확도: {:.2f}".format(mlp.score(X_train, y_train)))
print("테스트 세트 정확도: {:.2f}".format(mlp.score(X_test, y_test)))
```

```
훈련 세트 정확도: 0.94
테스트 세트 정확도: 0.92
```

데이터 전처리 후, 신경망 적용

- 모든 입력 특성을 평균은 0, 분산은 1로 변형

In [9]:
```
# 훈련 세트 각 특성의 평균을 계산합니다
mean_on_train = X_train.mean(axis=0)
# 훈련 세트 각 특성의 표준 편차를 계산합니다
std_on_train = X_train.std(axis=0)

# 데이터에서 평균을 빼고 표준 편차로 나누면
# 평균 0, 표준 편차 1 인 데이터로 변환됩니다.
X_train_scaled = (X_train - mean_on_train) / std_on_train
# (훈련 데이터의 평균과 표준 편차를 이용해) 같은 변환을 테스트 세트에도 합니다
X_test_scaled = (X_test - mean_on_train) / std_on_train

mlp = MLPClassifier(random_state=0)
mlp.fit(X_train_scaled, y_train)

print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_train_scaled, y_train)))
print("테스트 세트 정확도: {:.3f}".format(mlp.score(X_test_scaled, y_test)))
```

```
훈련 세트 정확도: 0.991
테스트 세트 정확도: 0.965
.../site-packages/sklearn/neural_network/_multilayer_perceptron.py:785: Conv
ergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and t
he optimization hasn't converged yet.
  warnings.warn(
```

스케일 조정으로 성능이 좋아짐.

위의 메시지가 나오면 반복횟수 늘리기

- Maximum iterations (200) reached and the optimization hasn't converged yet.

In [10]:
```
mlp = MLPClassifier(max_iter=1000, random_state=0)
mlp.fit(X_train_scaled, y_train)

print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_train_scaled, y_train)))
print("테스트 세트 정확도: {:.3f}".format(mlp.score(X_test_scaled, y_test)))
```

```
훈련 세트 정확도: 1.000
테스트 세트 정확도: 0.972
```

훈련세트와 테스트 세트의 차이가 있으므로 일반화 성능 향상시키기

In [11]:
```
mlp = MLPClassifier(max_iter=1000, alpha=1, random_state=0)
mlp.fit(X_train_scaled, y_train)

print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_train_scaled, y_train)))
print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_test_scaled, y_test)))
```

```
훈련 세트 정확도: 0.988
훈련 세트 정확도: 0.972
```

가중치 alpha 0.0001에서 1로 크게 증가시키기

In [12]:
```
mlp = MLPClassifier(max_iter=1000, alpha=1, random_state=0)
mlp.fit(X_train_scaled, y_train)

print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_train_scaled, y_train)))
print("훈련 세트 정확도: {:.3f}".format(mlp.score(X_test_scaled, y_test)))
```

```
훈련 세트 정확도: 0.988
훈련 세트 정확도: 0.972
```

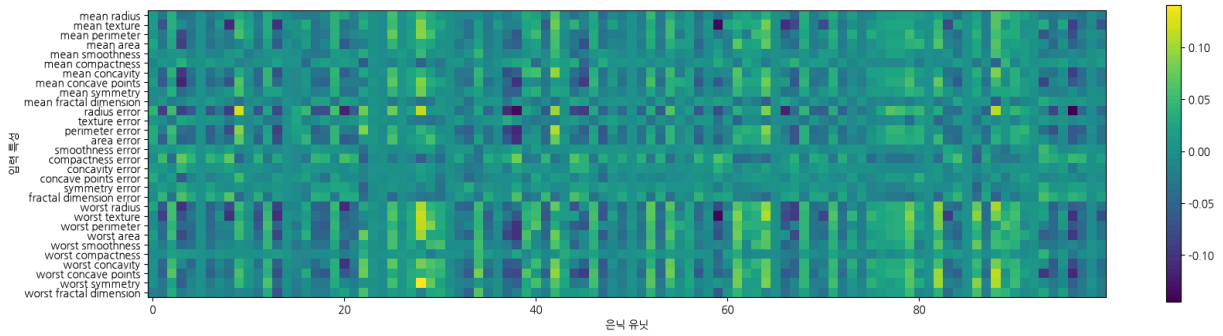
In [13]:
```
mlp.coefs_[0].std(axis=1), mlp.coefs_[0].var(axis=1)
```

Out[13]:
```
(array([0.02559193, 0.04066835, 0.03308025, 0.03081333, 0.01669964,
        0.01838658, 0.03626209, 0.04066151, 0.02432519, 0.01737967,
        0.05508384, 0.01659837, 0.04014836, 0.036805  , 0.0131538 ,
        0.03408178, 0.01182728, 0.015795  , 0.01424585, 0.02966082,
        0.04854174, 0.05185808, 0.04186009, 0.04332929, 0.03185674,
        0.01475534, 0.04066124, 0.04863656, 0.03751535, 0.0262638 ]),
 array([0.00065495, 0.00165391, 0.0010943 , 0.00094946, 0.00027888,
        0.00033807, 0.00131494, 0.00165336, 0.00059171, 0.00030205,
        0.00303423, 0.00027551, 0.00161189, 0.00135461, 0.00017302,
        0.00116157, 0.00013988, 0.00024948, 0.00020294, 0.00087976,
        0.0023563 , 0.00268926, 0.00175227, 0.00187743, 0.00101485,
        0.00021772, 0.00165334, 0.00236552, 0.0014074 , 0.00068979]))
```

In [14]:
```
import matplotlib
matplotlib.rcParams['axes.unicode_minus'] = False
```

In [15]:
```
plt.figure(figsize=(20, 5))
plt.imshow(mlp.coefs_[0], interpolation='none', cmap='viridis')
plt.yticks(range(30), cancer.feature_names)
plt.xlabel("은닉 유닛")
plt.ylabel("입력 특성")
plt.colorbar()
```

Out[15]:
```
<matplotlib.colorbar.Colorbar at 0x11745a4b0>
```



모델에서 작은 가중치를 가진 특성은 모델에 덜 중요하다.

- smoothness error, fractal dimension error사이의 특성
- mean smoothness와 meancompactness가 다른 특성들보다 가중치가 낮다

장단점과 매개변수

장점

- 주요한 장점은 대량의 데이터의 내재된 정보를 잡아내고 매우 복잡한 모델을 만드는 것이 가능
- 충분한 연산 시간과 데이터를 주고, 매개변수를 세심하게 조정하면 신경망은 뛰어난 성능을 낸다.

단점

- 신경망은 종종 학습이 오래 걸린다.
- 다른 종류의 특성을 가진 데이터라면 트리 기반 모델이 더 잘 작동

매개변수

- 가장 중요한 매개변수는 은닉층의 개수와 각 유닛층의 유닛 수
- 처음에 한개 또는 두개의 은닉층으로 시작해서 늘려가기
- 유닛수는 보통 입력 특성의 수와비슷하게 설정하지만 수천 초중반을 넘는 일은 거의 없음.

신경망의 매개변수를 조정하는 일반적인 방법은 먼저 충분히 과대적합되어, 문제를 해결할만큼의 큰 모델을 만든다. 이후 충분히 학습될 수 있다고 생각할 때, 신경망의 구조를 줄이거나 규제 강화를 위해 alpha값을 증가시켜 일반화 성능을 향상시킴.

In [ ]: